

CampaignOS

AI-Powered Campaign Production Platform

Agent Architecture & Technical Reference

Platform: Google ADK 2.0 + Vertex AI

Models: Gemini 3.5 Flash . Gemini 3 Pro Image . Veo 3.1

Infra: Google Cloud Run . GCS . BigQuery . pgvector

Frontend: React 18 . TypeScript . Vite

Date: June 2026

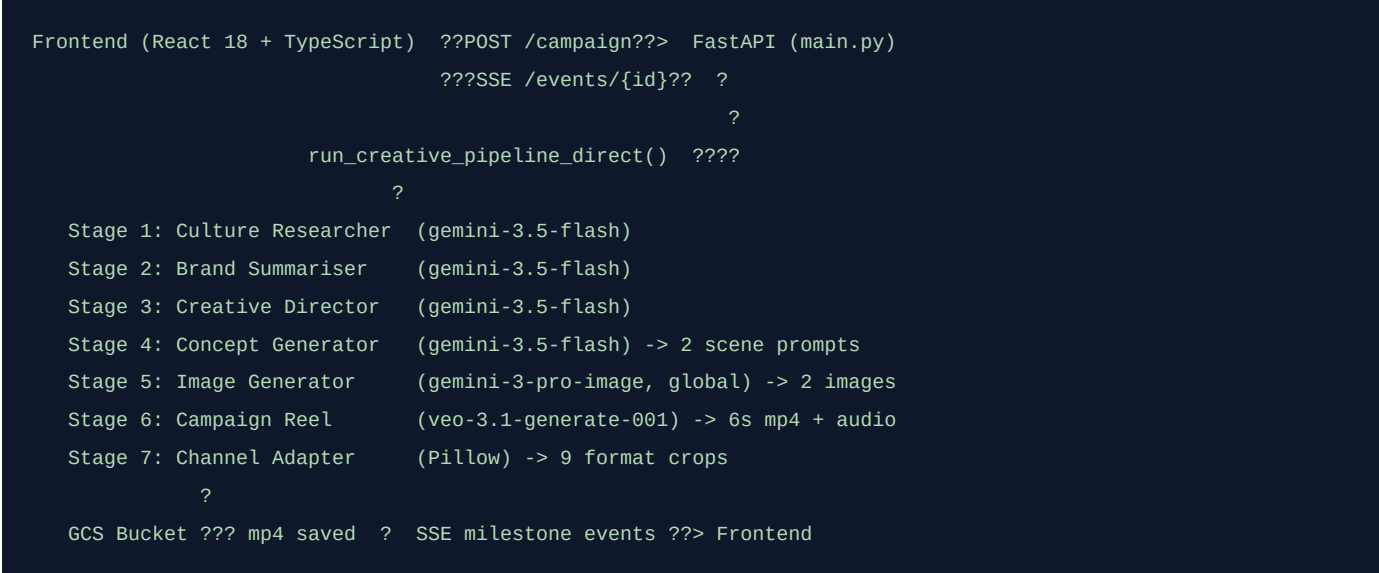
Table of Contents

- 1. System Overview 3
- 2. PATH 1 -- ADK Workflow DAG (pipeline.py) 4
 - 2.1 load_brand_context (Function Node) 4
 - 2.2 briefing_agent 4
 - 2.3 culture_analyst 5
 - 2.4 culture_formatter 5
 - 2.5 creative_director 5
 - 2.6 copy_agent 6
 - 2.7 kv_generator_1 / kv_generator_2 6
 - 2.8 kv_image_agent_1 / kv_image_agent_2 7
 - 2.9 kv_ranker + hitl_kv_selection 7
 - 2.10 channel_router -> performance_agent 8
- 3. PATH 2 -- Direct Pipeline (runner.py) 9
 - 3.1 Culture Researcher 9
 - 3.2 Brand Summariser 9
 - 3.3 Creative Director 10
 - 3.4 Concept Generator (2 scene concepts) 10
 - 3.5 Image Generator -- Gemini 3 Pro Image 11
 - 3.6 Campaign Reel -- Veo 3.1 12
 - 3.7 Channel Adaptations (9 formats) 12
- 4. Supporting Components 13
- 5. Files Modified / Added 14
- 6. Model Reference 15
- 7. Cloud Run Deployment 15

1. System Overview

CampaignOS is a multi-agent AI platform for automated campaign production. A 7-step wizard (Brand -> Goal -> Product -> Fan Truth -> Audience -> Channels/KPIs -> Budget) collects the campaign brief. The backend orchestrates a pipeline of AI agents that produce a validated brief, cultural intelligence, creative strategy, campaign copy, key visual images, a 6-second video reel, and channel-specific adaptations -- all streamed live to the UI via SSE.

Architecture Diagram



2. PATH 1 -- ADK Workflow DAG

Defined in `app/pipeline.py` + `app/agents.py`. Used by `/brief` and `/pipeline` routes. Google ADK 2.0 Workflow resolves the DAG edges, handles fan-out/fan-in parallelism automatically. Function nodes run at zero LLM cost.

load_brand_context

Stage 1 . Function Node (zero LLM cost)

Files: `app/nodes.py` `app/brand_assets.py`

Model: `None` -- deterministic Python

Loads brand guidelines, product map, font paths, and benchmark data from GCS (production) or local bucket/brands/ (development). Injects brand context as session state placeholders consumed by all downstream agents.

- ? Reads `brand_guidelines.md`, product catalogue, font .ttf files, benchmark JSON
- ? Supports `BRAND_ASSETS_MODE=gcs` (Cloud Storage) or local (bucket/ directory)
- ? Session state keys: `{brand_guidelines}`, `{brand_name}`, `{product_map}`
- ? Zero LLM calls -- purely deterministic file reads

briefing_agent

Stage 2 . Briefing Stage

Files: `app/agents.py` `app/instructions.py` `app/models.py`

Model: `gemini-3.5-flash` (global endpoint)

Validates and enriches the incoming campaign brief. Scores Fan Truth relevance, flags KPIs against historical benchmarks, and produces a structured MachineBrief artifact.

- ? Input: BriefRequest (brand, product, fan_truth, audience, channels, KPIs, budget)
- ? Output: MachineBrief -- validated, enriched brief with `fan_truth_score` (0-100)
- ? Writes to session state via `output_key='machine_brief'`
- ? `mode=single_turn` -- one LLM call, enforced output schema

culture_analyst

Stage 3 . Cultural Intelligence (multi-turn)

Files: `app/agents.py` `app/instructions.py`

Model: `gemini-3.5-flash` + Google Search tool

Researches the cultural landscape surrounding the campaign using live Google Search. Multi-turn so it can iterate: search -> read -> refine -> search again across multiple queries.

- ? Tool: `google_search` (ADK built-in, live web search)
- ? NOT `single_turn` -- multiple tool calls for thorough research
- ? Produces raw cultural intelligence: trends, tensions, moments relevant to audience
- ? Followed immediately by `culture_formatter` to structure the output

culture_formatter

Stage 4 . Culture Structuring

Files: app/agents.py app/instructions.py app/models.py

Model: [gemini-3.5-flash](#) (global endpoint)

Takes raw cultural research from culture_analyst and structures it into a typed CultureAnalysis Pydantic object with defined fields.

- ? output_schema=CultureAnalysis (enforces JSON structure)
- ? Single-turn -- one formatting call
- ? Saved to session state + artifact service via persist_culture function node

creative_director

Stage 5 . Creative Strategy

Files: app/agents.py app/instructions.py app/models.py

Model: [gemini-3.5-flash](#) (global endpoint)

Synthesises MachineBrief + CultureAnalysis + brand guidelines into a full CreativeStrategy with Big Idea, visual world, hero message, and creative tension.

- ? Inputs: MachineBrief, CultureAnalysis, brand_guidelines (from session state)
- ? Output: CreativeStrategy -- Big Idea title (?6 words), Visual World (2-3 sentences),
- ? Hero Message (?8 words, Fan-to-Fan voice), Creative Tension (cultural hook)
- ? output_schema=CreativeStrategy, mode=single_turn

copy_agent

Stage 6 . Campaign Copy

Files: app/agents.py app/instructions.py app/models.py

Model: [gemini-3.5-flash](#) (global endpoint)

Distills the creative strategy into short, medium, and long copy variants plus channel-specific copy for every selected channel.

- ? Short: billboard headline (?8 words)
- ? Medium: social media headline (?20 words)
- ? Long: email/web body copy (100-150 words)
- ? Channel copy: Instagram caption, TikTok hook, Google Ads RSA, email subject, OOH
- ? output_schema=CampaignCopy, mode=single_turn

kv_generator_1 + kv_generator_2

Stage 7 . KV Art Direction (parallel fan-out)

Files: app/agents.py app/instructions.py

Model: **gemini-3.5-flash** (global endpoint)

Two parallel art directors with deliberately different design philosophies. Each produces a KVConcept with a detailed Gemini Pro Image generation prompt.

- ? kv_generator_1 -- GRAPHIC-LED: layout architecture first, colour fields, typographic structure, grid, negative space; photography in service of the graphic system
- ? kv_generator_2 -- IMAGE-LED: single powerful photographic moment carries the emotional weight; typography designed into the composition, not placed after
- ? Output keys: kv_concept_1 and kv_concept_2 written to session state
- ? Fan-out: both run in parallel from persist_copy via separate DAG edges

kv_image_agent_1 + kv_image_agent_2

Stage 8 . Image Generation (parallel)

Files: app/agents.py app/tools.py app/instructions.py

Model: **gemini-3.5-flash** (reasoning) + **gemini-3-pro-image** (image, global endpoint)

Generates the hero KV image for each concept using Gemini 3 Pro Image via the generate_and_save_kv_image tool. Saves to ADK artifact service.

- ? Tool: generate_and_save_kv_image -- calls gemini-3-pro-image via generate_content()
- ? API: response_modalities=['IMAGE','TEXT'] + ImageConfig(aspect_ratio='16:9')
- ? Multimodal inputs: product reference images + logo passed alongside text prompt
- ? Artifact saved to InMemory (dev) or GCS (production) via ADK artifact service
- ? Fan-out parallel: kv_image_agent_1 and kv_image_agent_2 run simultaneously

aggregate_kv_concepts -> kv_ranker -> hitl_kv_selection

Stage 9 . Fan-In + Ranking + HITL

Files: app/nodes.py app/agents.py

Model: **kv_ranker: gemini-3.5-flash** | **aggregate: function node** (zero LLM)

Fan-in after both image agents complete. Merges concepts, ranks them, then suspends for human selection.

- ? JoinNode: waits for both kv_image_agent_1 and kv_image_agent_2 to complete
- ? aggregate_kv_concepts: function node -- merges both enriched concepts (zero LLM cost)
- ? kv_ranker: evaluates composition quality, brand alignment, fan truth resonance
- ? hitl_kv_selection: HITL gate -- pipeline suspends until human selects concept 1 or 2

channel_router -> content_agent -> execution_agent -> aggregation_agent -> performance_agent

Files: app/agents.py app/instructions.py

Model: **gemini-3.5-flash** (all agents, global endpoint)

Downstream pipeline from approved KV concept to final campaign package and performance framework.

- ? channel_router: maps approved KV + selected channels to per-channel format specs
- ? content_agent: generates production-ready copy + image prompts for every channel
- ? execution_agent: stub activation layer -- produces execution notes per platform
- ? aggregation_agent: consolidates all outputs into one CampaignAggregation record
- ? performance_agent: generates KPI tracking framework and first-48h monitoring plan

3. PATH 2 -- Direct Pipeline (/campaign route)

Defined in app/runner.py -> run_creative_pipeline_direct(). This is the primary path used by the frontend /campaign endpoint. Bypasses the ADK Workflow DAG entirely. All stages run sequentially, with SSE events pushed to the frontend at each step.

Culture Researcher

Stage 1

Files: app/runner.py

Model: [gemini-3.5-flash](#) (global endpoint)

Writes a 300-word cultural intelligence brief about what the target audience is feeling/experiencing right now -- cultural tensions, trends, and what the brand can tap into.

- ? Inputs: brand, audience (segment, age, interests, market)
- ? Output: free-text cultural brief (300 words max)
- ? SSE event: culture/done with brief preview
- ? Runs once per campaign via _llm() helper

Brand Summariser

Stage 2

Files: app/runner.py

Model: [gemini-3.5-flash](#) (global endpoint)

Reads the brand guidelines (up to 3,500 characters) and distils exactly 5 numbered brand lock points that every creative execution must honour.

- ? Includes exact HEX colours (Sunglow #B00064/#FFC72C, Rnorr #008641/#FFDE00, Boozt #0E105E/#0086FE)
- ? Covers: colours, typography/font, logo rules, tone, forbidden treatments
- ? SSE event: kv/step_data with brand_locks preview
- ? Hardcoded palette lock per brand as fallback

Creative Director

Stage 3

Files: app/runner.py

Model: [gemini-3.5-flash](#) (global endpoint)

Synthesises culture + brand locks + big_idea_seed into a Big Idea for the campaign.

- ? Output: Big Idea title (?6 words), Visual World (2-3 sentences), Hero Message (?8 words),
- ? Creative Tension (cultural hook sentence)
- ? SSE event: kv/step_data with big_idea preview
- ? big_idea_seed from strategy.hero_message passed in from wizard

Concept Generator

Stage 4

Files: app/runner.py

Model: [gemini-3.5-flash](#) (global endpoint)

Generates 2 distinct scene concept prompts using all brief inputs. Each concept is a 170-200 word image generation prompt with different mood, setting, and energy.

- ? Inputs: brand, product, big_idea, fan_truth, audience, season, market
- ? Brand-specific magic effects per brand (Sunglow: golden particles/hair halo;
- ? Rnorr: steam/golden food particles; Boozt: electric blue trails/neon arcs)
- ? Concept 1 -- DYNAMIC: model in full motion, maximum magical effects
- ? Concept 2 -- INTIMATE GLOW: close-up, intense eye contact, deeply saturated
- ? SSE event: kv/step_data with image_prompt (first concept preview)
- ? Concepts parsed via regex from [CONCEPT 1 - DYNAMIC]: and [CONCEPT 2 - INTIMATE]: markers

Image Generator

Stage 5

Files: app/runner.py app/creative_pipeline.py

Model: [gemini-3-pro-image](#) (global endpoint, GA since 2026-05-27)

Generates 2 full-bleed 16:9 landscape advertising images in parallel, one per concept. Uses actual product images and logo as multimodal visual reference inputs.

- ? API: client.models.generate_content() with response_modalities=['IMAGE','TEXT']
- ? Config: GenerateContentConfig + ImageConfig(aspect_ratio='16:9')
- ? Multimodal inputs: logo (most important anchor) + up to 3 product reference images
- ? Brand asset style analysis: gemini-3.5-flash analyzes existing campaign imagery
- ? -> extracts magic effects, model energy, gradients -> appended as style suffix
- ? No-text rule: 'Do NOT render ANY text except product packaging labels'
- ? Brand + product name enforced: 'Label MUST read Rnorr and Rnorr Chicken Cubes'
- ? Staggered generation: 5s delay between calls to avoid 429 rate limits
- ? Retry: exponential backoff 20s -> 40s -> 80s on 429
- ? Pillow overlay: billboard text (accent colour word 1, white rest), tight 1px outline,
- ? brand logo top-right with soft glow, no vignette panel

Campaign Reel Generator

Stage 6

Files: app/runner.py -> generate_campaign_reel()

Model: [gemini-3.5-flash](#) (prompt) + [veo-3.1-generate-001](#) (video+audio, global endpoint)

Generates a 6-second 16:9 campaign reel with native audio, background music, and voiceover reading the campaign headline.

Product-specific scene directions per brand.

- ? Product-aware scenes: Sunglow (serum=drop application / conditioner=bathroom / shampoo=hair flip)
- ? Rnorr (gravy=pouring over pan / bouillon=sprinkling into pot / cubes=dissolving in broth)
- ? Boozt (mousse/spray=root lift explosion / shampoo=volume reveal)
- ? Veo config: duration_seconds=6, aspect_ratio='16:9', generate_audio=True, number_of_videos=1
- ? Voiceover: 'A warm confident voiceover says: [copy_headline]' -- same text as KV image
- ? Operation polling: every 20s, max 8 minutes timeout
- ? Video saved to gs://{bucket}/outputs/{campaign_id}/reel.mp4
- ? Downloaded from GCS, returned as base64 mp4
- ? Controlled by REEL_ENABLED env var (set false to skip and save ~5 min)
- ? Model configurable via VEO_MODEL env var

Channel Adapter

Stage 7

Files: app/runner.py -> _create_channel_adaptation()

Model: [None](#) -- [Pillow image processing only](#) (zero LLM cost)

Crops and adapts the primary KV image into 9 channel-specific formats. Only generates formats relevant to the channels selected in the wizard.

- ? Instagram Feed: 1:1 square
- ? Instagram Stories + TikTok: 9:16 vertical
- ? YouTube + Google Ads + Website Banner: 16:9 landscape
- ? Meta Ads: 1:1 square
- ? Email Banner: 3:1 wide
- ? OOH / Billboard: 4:1 ultra-wide
- ? Stored in channel_adaptations dict, returned in result and pushed via SSE milestone
- ? Website Banner (16:9) used as landing page hero background

4. Supporting Components

Campaign Email

Publisher -- send_campaign_email()

Files: app/publisher.py

Model: **None** -- SMTP via Gmail / SendGrid / Mailgun

Sends a branded HTML campaign email with KV image, logo, headline, copy, and CTA. Handles Gmail's image loading restrictions automatically.

- ? Localhost detection: if HARNESS_URL contains localhost, uses base64 inline images
- ? (Gmail cannot fetch from localhost -- images would be blank)
- ? Production: serves images via hosted URLs (/campaign-image/{id}, /brand-logo/{brand})
- ? Template: premium HTML email with brand colours, product spotlight, landing page CTA
- ? Logo loaded from local bucket/ or GCS via _get_logo_b64()

Landing Page Generator

Publisher -- generate_brand_website()

Files: app/publisher.py

Model: **None** -- Python HTML generation (Jinja-free)

Generates a full brand website HTML page for Rnorr, Sunglow, or Boozt. Each brand has its own generator with brand-specific design.

- ? Hero background: website banner (16:9 channel adaptation) > brand asset > gradient fallback
- ? KV image (with billboard text) shown only in Campaign section -- not in hero
- ? Rnorr recipe cards: distinct gradients (warm orange / rich amber / lime-yellow),
- ? different food emojis at large scale, Quick & Easy / Comfort Food / Family Favourite tags
- ? Persistent URL: stored in localStorage by campaign_id, shown as green banner
- ? 'Open Website ->' button survives navigation and page reloads

pgvector CDP Client

CDP -- pgvector_client.py

Files: app/pgvector_client.py

Model: **gemini-embedding-2** (768-dim embeddings, Vertex AI)

PostgreSQL + pgvector semantic search over customer data profiles (CDP). Provides audience insights, brand guidelines, fan truths, and benchmarks.

- ? search_brand_guidelines(): semantic search over brand guidelines
- ? search_fan_truths(): matches fan truth to brand/product/category
- ? search_campaign_benchmarks(): historical performance by brand/market/season
- ? search_audience_insights(): demographic + psychographic profiles
- ? search_channel_benchmarks(): channel-specific performance metrics
- ? Embeddings: USE_GEMINI_EMBEDDINGS=true uses gemini-embedding-2 (768-dim)

5. Files Modified / Added

harness/app/config.py **MODIFIED** Added AliasChoices for GOOGLE_CLOUD_LOCATION -> gcp_region
harness/app/agents.py **CORE** ADK 2.0 agent definitions -- all 14 agents
harness/app/pipeline.py **CORE** Workflow DAG edges -- briefing_pipeline + root_agent
harness/app/runner.py **CORE** Direct pipeline: 7 stages, Veo reel, brand overlay, channels
harness/app/instructions.py **CORE** All agent system prompts (1400+ lines)
harness/app/nodes.py **CORE** Function nodes: load_brand_context, persist_*, aggregate_kv
harness/app/tools.py **CORE** generate_and_save_kv_image ADK tool
harness/app/models.py **CORE** Pydantic models: BriefRequest, MachineBrief, CreativeStrategy
harness/app/publisher.py **MODIFIED** Landing pages (3 brands) + email template + hero fix
harness/app/brand_assets.py **CORE** GCS local asset loader abstraction
harness/app/pgvector_client.py **CORE** pgvector CDP semantic search (5 functions)
harness/app/search_client.py **CORE** Vertex AI Search wrapper
harness/app/data_loader.py **CORE** BigQuery audit logger
harness/main.py **MODIFIED** FastAPI routes, SSE, publish, image/logo serve endpoints
harness/cloudbuild.yaml **MODIFIED** Cloud Run: models, global location, Veo vars, timeout 3600s
harness/Dockerfile **CORE** Python 3.12, uv, Pillow deps, brand assets at build time
harness/.env **LOCAL** dev env vars (not committed -- gitignored)
frontend/src/App.tsx **MODIFIED** 7-step wizard, KV panel, reel player, landing URL persistence
frontend/src/hooks/usePipeline.ts **CORE** SSE stream parsing, pipeline state management
frontend/src/types/pipeline.ts **CORE** TypeScript interfaces for harness API
frontend/cloudbuild.yaml **CORE** Cloud Run frontend: nginx, VITE_API_URL build arg

6. Model Reference

Model ID	Location	Used For	Stage
gemini-3.5-flash	global	All text reasoning agents -- brief, culture, strategy, copy, concept generation	GA
gemini-3-pro-image	global	KV image generation -- 16:9 landscape, multimodal product references	GA
veo-3.1-generate-001	global	6s campaign reel -- native audio, voiceover, brand-specific cinematic scenes	GA
gemini-embedding-2	global	768-dim embeddings for pgvector CDP semantic search	GA
groq/llama-3.3-70b	LiteLLM	Fallback reasoning via Groq free tier (14,400 req/day)	FALLBACK

7. Cloud Run Deployment

Harness service: campaignos-harness . 4GB RAM . 2 vCPU . timeout 3600s . concurrency 10

Frontend service: campaignos-frontend . 256Mi RAM . 1 vCPU . Nginx static serving

Region: us-central1 (Cloud Run service) | global (Vertex AI model endpoints)

Build trigger: gcloud builds submit --config=cloudbuild.yaml

Secrets: GROQ_API_KEY . GOOGLE_EMBEDDING_KEY . EMAIL_APP_PASSWORD . EMAIL_FROM

Cloud SQL: campaignos-pgvector (PostgreSQL + pgvector, us-central1)

GCS Bucket: dauntless-karma-497108-b0-campaignos (brand assets + pipeline outputs)

Project ID: dauntless-karma-497108-b0